DATA STRUCTURES

ETCCDS202

Assignment-4

Topic: Non-Linear and Advanced Data Structure (Unit-4)

Short Analysis Report

Submitted By: Meghna Kumar
Roll No.: 2501730214
Program: B.Tech CSE (AI & ML)
Section: A
Faculty Name: Ms Neetu Chauhan
Submitted on: 17-04-2026

Assignment Overview:

In this assignment, I developed a Data Management Mini Toolkit (DMMT) which uses the programming language Python to showcase how important non-linear data structures work. This toolkit contains three different data structures, such as a Binary Search Tree (BST) that enables us to store data in order, a Graph that utilizes an adjacency list and traversals, including BFS and DFS, and a Hash Table with separate chaining for efficient key-value storage and collision handling.

Why inorder traversal prints a BST in sorted order?

In Binary Search Trees (BST):

Elements in the left sub-tree are always less than the root element.

Elements in the right sub-tree are always greater than the root element.

An In-order Traversal will follow the sequence as: Left - Root - Right

Thus, it visits elements lesser than the node first, then the node itself, and lastly elements greater than the node.

Time complexity discussion in simple terms (average O(log n), worst O(n) if skewed):

The efficiency of inserting, searching, or deleting a value from a Binary Search Tree is determined by the height of the tree.

Average Time Complexity: O(log n)

For a balanced BST, each step halves the number of elements to be searched, just like binary search, leading to faster operations in logarithmic time.

Worst Time Complexity: O(n)

For an unbalanced BST, where the tree is skewed in the form of a linked list, we might need to traverse all elements of the tree, leading to slower operations taking linear time.

In short, BST is very efficient when balanced but not so efficient when skewed.

Difference between BFS and DFS:

- BFS (Breadth-First Search):
  Exploration is done layer-by-layer in BFS algorithm. BFS visits all nodes at one level before moving to the next level. BFS algorithm makes use of a queue.
- DFS (Depth-First Search):
  In DFS algorithm, the graph is explored to its maximum depth before going to the backtracking step. One path is fully searched before starting another path.
- In brief:
  - BFS = wide search (level-wise)
  - DFS = deep search (path-wise)

One real-world use case each (e.g., BFS for shortest path in unweighted graph, DFS for cycle detection):

- **BFS (Breadth-First Search):**
  Used in web crawling and social media connections, where platforms explore all immediate connections (friends/followers) first before moving to the next level of connections.
- **DFS (Depth-First Search):**
  Used in solving puzzles like mazes or backtracking problems, where the algorithm explores one path completely until it reaches a dead end, then backtracks to try other paths.

What is collision and why it happens?

A collision in a hash table occurs when two or more different keys are mapped to the same index using a hash function.

It happens because:

- The hash table has a limited number of slots, but there can be many keys.
- The hash function (e.g., key % table_size) can produce the same result for different keys.

Why separate chaining helps?

Separate chaining helps by allowing multiple elements to be stored at the same index using a list (or chain).

Instead of overwriting data during a collision, all elements that hash to the same index are stored together in that bucket.

This ensures that:

- No data is lost during collisions
- Operations like insert, search, and delete can still be performed correctly
- The hash table remains efficient even when collisions occur

Time Complexity of Hash Table (Separate Chaining):

- Average Case: O(1)
  When the hash function distributes keys evenly, each bucket has very few elements. So, operations like insert, search, and delete take constant time.
- Worst Case: O(n)
  If many collisions occur and all elements go into the same bucket, the structure behaves like a list. In this case, operations may require checking all elements, taking linear time.
- In short, hash tables are very fast on average, but performance can degrade if too many collisions happen.

Conclusion:

This assignment helped in understanding the implementation and working of important non-linear data structures like Binary Search Trees, Graphs, and Hash Tables. It demonstrated how these structures support efficient searching, traversal, and data storage. By implementing them in Python, the concepts became clearer, along with their real-world applications and performance. Overall, this assignment strengthened practical knowledge of data structures used in modern computing systems.